Tutoriumsblatt 1

Git, Tests und Teamarbeit

Bearbeitungszeit etwa 90 Minuten · 40 Punkte · Lösungen im Anhang ab Seite 6

Nach diesem Blatt kannst du

- einen Commit-Graphen, einen Dreiwege-Merge und das Rückgängigmachen von Hand vorhersagen,
- Tests mit Äquivalenzklassen, Grenzwerten und Mutanten bewerten, ein CI-Protokoll lesen und freundlich reviewen.

So arbeitest du mit dem Blatt

Du brauchst keinen Rechner, nur Papier und Stift. Alle Befehle stehen wie im Skript (Git 2.43, uv 0.13.0, pytest 9.1.1, ruff 0.17.0), Editoren schließt du mit der vorgeschlagenen Nachricht. Commits heißen hier nach Buchstaben statt nach IDs. Hängst du, rechne mit den Werten unter „Zur Kontrolle“ weiter. „Ausgedacht“ heißt fiktiv. Zum Notebook: Aufgabe 1 gehört zu TODO 2 und 3; Aufgabe 2 zu 4; Aufgabe 3 zu 5 bis 8 und 11; Aufgabe 4 zu den Schritten L6 und L9 „Auf deinem Laptop“; Aufgabe 5 zu L12 und L13.

Aufgabe 1 Commit-Graph von Hand

9 Punkte · etwa 20 Min.

Bezug: Modul 1, Lerneinheiten 2 (Branches, Merge und Konflikte) und 3 (Zusammenarbeit über GitHub)

Am Anfang zeigen `main` und `origin/main` auf Commit A, du bist auf `main`. Vor jedem `git commit` ist eine passende Änderung vorgemerkt, der Kommentar nennt den neuen Commit. Merge-Commits heißen M (Zeile 9) und N (Zeile 11), falls einer entsteht. Zwischen Zeile 9 und 10 pusht Kim den Commit E „Beschreibung ergänzt“ (Elternteil A, nur `pyproject.toml`).

```
1  git switch -c fix/illinois
2  git commit -m "Neue Schichtnamen erkennen"     # B
3  git commit -m "Regressionstest für Issue 1"    # C
4  git switch main
5  git switch -c doku/nachlieferung
6  git commit -m "README nennt Nachlieferung"     # D
7  git switch main
8  git merge doku/nachlieferung
9  git merge fix/illinois
10 git fetch
11 git pull --no-rebase
12 git push
13 git revert --no-edit doku/nachlieferung       # R
```

(a) Zeichne den Graphen nach Zeile 13: jeden Commit als Kreis mit Buchstaben, Pfeile zu den Eltern (bei Merge-Commits den ersten Elternteil kennzeichnen), dazu die Zeiger `main`, `origin/main`, `fix/illinois`, `doku/nachlieferung` und `HEAD`. Zur Kontrolle: Der Graph hat 8 Commits. [3 P]

(b) Bestimme für die Merges in den Zeilen 8, 9 und 11 die Merge-Basis und ob Git vorspulen kann (Fast-Forward). Könnte `fix/illinois` nach Zeile 13 per Fast-Forward auf den Stand von `main` rücken? Begründe mit der Merge-Basis. [2 P]

(c) Wie viele Commits ist `main` gegenüber `origin/main` vorne und hinten nach den Zeilen 9, 10, 12 und 13? Warum weiß `git status` nach Zeile 9 noch nichts von Kims Commit? Zur Kontrolle: Nach Zeile 10 sind es zusammen 5. [2 P]

(d) Welche Commits zeigt `git log --oneline --graph` nach Zeile 13, welcher steht oben, welche zeigt Git als Merge-Commit? Welche Commits zeigt `git log --oneline fix/illinois`, und zeigt der erste Befehl mit `--all` mehr? [2 P]

Aufgabe 2 Dreiwege-Merge von Hand

7 Punkte · etwa 15 Min.

Bezug: Modul 1, Lerneinheiten 2 (Branches, Merge und Konflikte) und 6 (Automatisch prüfen: pre-commit und CI)

```
# pyproject.toml, Basis
description = "Importer fuer beide Werke"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []

[dependency-groups]
dev = [
    "mypy>=2.4.0",
    "ruff>=0.17.0",
]
```

Ausschnitte aus `pyproject.toml` eines Übungsprojekts ab der Beschreibung. Auf `main` hast du mit `uv add --dev pytest` pytest aufgenommen, die Einstellungen für pytest angehängt und „fuer“ korrigiert. Kim hat auf `feature/hooks` mit `uv add --dev pre-commit` pre-commit aufgenommen, `requires-python` eingeschränkt und denselben Tippfehler korrigiert. Jetzt führst du auf `main` `git merge feature/hooks` aus. Git vergleicht Zeile für Zeile; was beide Seiten an derselben Stelle einfügen, ist eine Änderung dieser Stelle.

```
# unsere Seite: main
description = "Importer für beide Werke"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []

[dependency-groups]
dev = [
    "mypy>=2.4.0",
    "pytest>=9.1.1",
    "ruff>=0.17.0",
]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

```
# ihre Seite: feature/hooks
description = "Importer für beide Werke"
readme = "README.md"
requires-python = ">=3.13,<3.14"
dependencies = []

[dependency-groups]
dev = [
    "mypy>=2.4.0",
    "pre-commit>=4.7.0",
    "ruff>=0.17.0",
]
```

(a) Geh jede Stelle durch, an der mindestens eine Seite von der Basis abweicht: Wer hat sie geändert, und übernimmt Git sie automatisch? [2 P]

(b) Schreib den Ausschnitt so hin, wie Git ihn nach dem Merge mit Konfliktmarkern in die Datei schreibt. Zur Kontrolle: Er hat dann 18 Zeilen, Marker eingeschlossen. [2 P]

(c) Löse den Konflikt fachlich richtig und begründe. Git meldet auch in `uv.lock` Konflikte. Welcher Weg ist richtig: (1) die Marker in `uv.lock` von Hand auflösen, (2) `uv.lock` löschen und `uv lock`, (3) `git checkout --ours uv.lock` (stellt die Fassung von `main` her), dann `uv lock`? Mit welchen Befehlen schließt du den Merge ab? [3 P]

Aufgabe 3 Tests entwerfen und bewerten

10 Punkte · etwa 22 Min.

Bezug: Modul 1, Lerneinheiten 4 (Tests mit pytest) und 5 (Fixtures, Testdaten und Regressionstests)

Kim ordnet Störmeldungen der Instandhaltung im Stammwerk einer Schicht zu (ausgedacht): Paket `stoermeldung` im Repository `nordwerk-stoermeldungen`, mit derselben CI wie deine, nur

ohne die Schritte für die Lieferungen. Oben `src/stoermeldung/schicht.py`, darunter links Kims Tests `tests/test_schicht.py`.

```python
import datetime as dt


def schicht(uhrzeit: str) -> str:
    """Schicht zur Uhrzeit "HH:MM": Früh 06:00 bis 13:59, Spät 14:00 bis 21:59.

    Sonst ValueError mit "keine Schicht"; unlesbar: ValueError von dt.time.
    """
    zeit = dt.time.fromisoformat(uhrzeit)
    if dt.time(6) <= zeit < dt.time(14):
        return "Früh"
    if dt.time(14) <= zeit < dt.time(22):
        return "Spät"
    raise ValueError(f"keine Schicht um {uhrzeit}")
```

```python
"""Tests für stoermeldung.schicht."""

import pytest

from stoermeldung.schicht import schicht


@pytest.mark.parametrize(
    ("uhrzeit", "erwartet"),
    [
        ("09:30", "Früh"),
        ("14:00", "Spät"),
        ("21:59", "Spät"),
    ],
)
def test_schicht(uhrzeit, erwartet):
    assert schicht(uhrzeit) == erwartet


def test_nachts_keine_schicht():
    with pytest.raises(ValueError):
        schicht("23:10")


def test_vor_sechs_keine_schicht():
    with pytest.raises(ValueError, match="keine Schicht"):
        schicht("05:59")
```

(a) Bilde Äquivalenzklassen für `uhrzeit` und nenne die Grenzen. Gib eine kleinste Liste von Eingaben mit erwartetem Ergebnis an, die jede Klasse und jede Grenze von beiden Seiten prüft. Zur Kontrolle: Für die Grenzen allein brauchst du sechs Uhrzeiten. [2 P]

Die drei Fälle von `test_schicht` heißen T1 bis T3 (in der Reihenfolge der Liste), die beiden anderen Tests T4 und T5. Jeder Mutant unten ändert genau eine Zeile von `schicht`; `strptime` mit `"%H.%M"` erwartet etwa `09.30`.

Mutant	Zeile wird zu
M1	`if dt.time(6) < zeit < dt.time(14):`
M2	`if dt.time(6) <= zeit <= dt.time(14):`
M3	`if dt.time(14) <= zeit <= dt.time(22):`
M4	`raise ValueError(f"unlesbar: {uhrzeit}")`
M5	`zeit = dt.datetime.strptime(uhrzeit, "%H.%M").time()`

(b) Fülle eine Tabelle: Welcher der Tests T1 bis T5 wird bei welchem Mutanten rot? Welche Mutanten überleben, und welcher Test fehlt jeweils? Warum fängt T4 keinen Mutanten, obwohl M5 jede Uhrzeit unlesbar macht? Zur Kontrolle: Genau zwei Mutanten überleben. [4 P]

```python
def test_spaet_am_abend():
    schicht("17:45") == "Spät"


def test_frueh_am_morgen():
    assert schicht("07:15"), "Früh"
```

(c) Beide Tests links sind in pytest grün. Was prüfen sie wirklich? Nenne eine falsche Fassung von `schicht`, bei der beide grün bleiben, und schreib die Tests richtig. [2 P]

```python
# tests/test_meldungen.py, Importe wie oben
@pytest.fixture(scope="module")
def meldungen():
    return ["05:40", "06:10", "14:05"]


def test_nur_schichtzeiten(meldungen):
    meldungen.remove("05:40")
    assert [schicht(m) for m in meldungen] == ["Früh", "Spät"]


def test_drei_meldungen(meldungen):
    assert len(meldungen) == 3
```

Kim probiert die Datei lokal neben `test_schicht.py` aus, ohne sie zu committen. pytest führt die Tests einer Datei in der Reihenfolge aus, in der sie dort stehen.

(d) Was meldet pytest für die beiden Tests bei (1) `uv run pytest tests/test_meldungen.py`, (2) demselben Aufruf mit `-k drei`, (3) Aufruf (1) ohne `scope="module"`? Wie machst du die Tests unabhängig von ihrer Reihenfolge? [2 P]

Aufgabe 4 CI-Protokoll und Review

7 Punkte · etwa 16 Min.

Bezug: Modul 1, Lerneinheiten 3 (Zusammenarbeit über GitHub) und 6 (Automatisch prüfen: pre-commit und CI)

Die Werkleitung im Stammwerk legt fest (ausgedacht): Ab dem 2. November beginnt die Frühschicht um 5:30. Kim ändert dafür `schicht` aus Aufgabe 3 und öffnet den Pull Request „Frühschicht ab 5:30“ mit „Fixes #4“. Der erste Lauf der CI ist rot. Unten steht ab der Formatierung je Schritt der Befehl mit `$` davor und seine Ausgabe; `...` markiert Kürzungen.

```
$ uv run ruff format --check
4 files already formatted
$ uv run ruff check
All checks passed!
$ uv run mypy src
Success: no issues found in 2 source files
$ uv run pytest
============================= test session starts ==============================
...
collected 5 items

tests/test_schicht.py ....F                                              [100%]

=================================== FAILURES ===================================
_________________________ test_vor_sechs_keine_schicht _________________________

    def test_vor_sechs_keine_schicht():
>       with pytest.raises(ValueError, match="keine Schicht"):
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       Failed: DID NOT RAISE ValueError

tests/test_schicht.py:26: Failed
=========================== short test summary info ============================
FAILED tests/test_schicht.py::test_vor_sechs_keine_schicht - Failed: DID NOT ...
========================= 1 failed, 4 passed in 0.02s ==========================
```

(a) Welcher Schritt ist rot, welcher Test in welcher Datei und Zeile, was hat er erwartet und was ist passiert? Liefen die Schritte davor, und wäre `uv run pytest` gelaufen, wenn `ruff check` rot gewesen wäre? Was ist zu tun? [3 P]

Kim pusht einen zweiten Commit, die CI wird grün. Unten alle Änderungen des Pull Requests als `git diff` (gekürzte Kopfzeilen, eine Zeile Kontext, bei `schicht.py` fünf).

```
--- a/.github/workflows/ci.yml
+++ b/.github/workflows/ci.yml
@@ -15,3 +15,3 @@ jobs:
           version: "0.13.0"
```

```
-      - run: uv sync --locked
+      - run: uv sync
       - run: uv run ruff format --check
--- a/src/stoermeldung/schicht.py
+++ b/src/stoermeldung/schicht.py
@@ -5,10 +5,11 @@ def schicht(uhrzeit: str) -> str:
     """Schicht zur Uhrzeit "HH:MM": Früh 06:00 bis 13:59, Spät 14:00 bis 21:59.

     Sonst ValueError mit "keine Schicht"; unlesbar: ValueError von dt.time.
     """
     zeit = dt.time.fromisoformat(uhrzeit)
-    if dt.time(6) <= zeit < dt.time(14):
+    print("Uhrzeit gelesen:", zeit)
+    if dt.time(5, 30) <= zeit < dt.time(14):
         return "Früh"
     if dt.time(14) <= zeit < dt.time(22):
         return "Spät"
     raise ValueError(f"keine Schicht um {uhrzeit}")
--- a/tests/test_schicht.py
+++ b/tests/test_schicht.py
@@ -22,6 +22 @@ def test_nachts_keine_schicht():
         schicht("23:10")
-
-
-def test_vor_sechs_keine_schicht():
-    with pytest.raises(ValueError, match="keine Schicht"):
-        schicht("05:59")
```

(b) Prüfe die Änderungen mit der Definition of Done aus Lerneinheit 3. Finde vier Probleme und schreib zu jedem einen Review-Kommentar an Kim: freundlich, konkret, mit Begründung und Vorschlag. [4 P]

Aufgabe 5 Versionen und Rückgängig machen

7 Punkte · etwa 15 Min.

Bezug: Modul 1, Lerneinheiten 1 (Git als Gedächtnis des Projekts) und 7 (Versionen, Releases und Rückgängig machen)

(a) Ein Jahr später steht `nordwerk` bei Version `1.3.2` (ausgedacht). Welche Version folgt jeweils, und mit welchem `uv version --bump ...` setzt du sie? (1) Summenzeilen mit einem Leerzeichen vor `TOTAL` werden jetzt erkannt statt als unlesbar verworfen. (2) Neuer Unterbefehl `nordwerk bericht`, alles Bisherige bleibt. (3) Die Option `--aus` von `nordwerk import` heißt jetzt `--ausgang`, `--aus` gibt es nicht mehr. (4) (1) und (2) kommen zusammen in ein Release. Zur Kontrolle: Genau eine Änderung erhöht die Hauptversion. [3 P]

Für jeden Befehl rechts neu: `main` und `origin/main` zeigen auf C. B hat in `nordwerk.toml` `ausgang = "export"` gesetzt (in A noch `"daten/aufbereitet"`) und soll zurück, C hat `README.md` eine Zeile angehängt. In `src/nordwerk/cli.py` gibt es eine Änderung, die noch nicht vorgemerkt ist. `git reset --soft` setzt nur den Branch zurück, Staging-Bereich und Arbeitsverzeichnis bleiben.

```
A  Importer aus Modul 0
B  Ausgabe nach export
C  README nennt den Aufruf   <- main, origin/main
```

```
1 git restore src/nordwerk/cli.py
2 git revert --no-edit HEAD~1
3 git reset --soft HEAD~1
4 git reset --hard HEAD~2
```

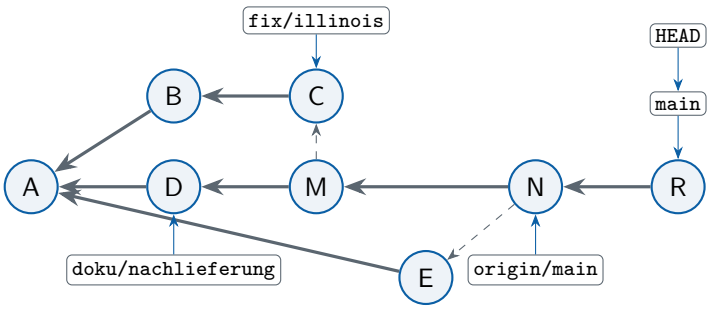
(b) Gib für jeden der vier Befehle an: Worauf zeigt `main` danach, wie viele Commits ist `main` vorne und hinten gegenüber `origin/main`, was steht in `nordwerk.toml` als `ausgang`, gibt es die Zeile aus C und die Änderung in `cli.py` noch, und wo? Welche Befehle sind sicher, wenn B und C schon gepusht sind? Zur Kontrolle: Nach genau einem Befehl ist `main` vorne. [4 P]

Anhang: Lösungen

Erst selbst rechnen, dann vergleichen. Die Lösungen zeigen den Rechenweg und typische Fehler.

Lösung zu Aufgabe 1 Commit-Graph von Hand

(a) Pfeile zeigen auf die Eltern, dick ist der erste Elternteil. Zeile 8 legt keinen Commit an.



8 Commits: A, B, C, D, E, M, N, R. M hat die Eltern D (erster, `main`) und C, N die Eltern M (erster) und E; R hat nur N. `HEAD` zeigt auf `main`, `main` auf R. `origin/main` steht auf N, denn Zeile 12 hat N gepusht, R noch nicht.

(b)

Zeile	Merge	Merge-Basis	Ergebnis
8	`main` (A) mit `doku/nachlieferung` (D)	A	Fast-Forward, `main` rückt auf D
9	`main` (D) mit `fix/illinois` (C)	A	kein Fast-Forward, Merge-Commit M
11	`main` (M) mit `origin/main` (E)	A	kein Fast-Forward, Merge-Commit N

Vorspulen geht genau dann, wenn die Merge-Basis der eigene Stand ist: Dann hat nur die andere Seite gearbeitet. Nach Zeile 13 ist die Merge-Basis von `fix/illinois` (C) und `main` (R) C selbst: C ist Vorfahr von R, `fix/illinois` ist 0 Commits vorne und 5 hinten (D, M, E, N, R). Also ja, `git merge main` auf `fix/illinois` spult vor.

(c)

nach Zeile	`origin/main`	vorne	hinten	`git status`
9	A	4	0	ahead by 4 commits (B, C, D, M)
10	E	4	1	diverged, 4 and 1 different commits
12	N	0	0	up to date
13	N	1	0	ahead by 1 commit (R)

`git status` vergleicht mit `origin/main`, also mit dem Stand von GitHub, den dein Rechner zuletzt gesehen hat. Erst `git fetch` in Zeile 10 holt E und rückt `origin/main` vor; bis dahin ist `origin/main` noch A. Nach Zeile 10 sind es $4 + 1 = 5$.

(d) Ohne `--all` zeigt Git alles, was von `HEAD` aus erreichbar ist: alle 8 Commits, oben R. Die beiden Merge-Commits M und N stehen mit `*` und darunter `|\`, wo die zweite Linie abzweigt. Die Reihenfolge der übrigen Zeilen verlangt die Aufgabe nicht. Mit `--graph` zeigt Git keinen Commit vor seinen Kindern und eine Linie möglichst am Stück; im Probe-Repository (Buchstaben statt IDs, Ort von GitHub gekürzt) sah es so aus, auch mit anderen Uhrzeiten der Commits:

```
* R Revert "README nennt Nachlieferung"
*   N Merge branch 'main' of ...
|\
| * E Beschreibung ergänzt
* |   M Merge branch 'fix/illinois'
|\ \
| * | C Regressionstest für Issue 1
| * | B Neue Schichtnamen erkennen
| |/
* / D README nennt Nachlieferung
|/
* A Importer aus Modul 0
```

`git log --oneline fix/illinois` zeigt C, B, A. `--all` zeigt nichts mehr: Jeder Branch und `origin/main` zeigen auf Commits, die schon von `main` aus erreichbar sind.

Typischer Fehler

In (c) nach Zeile 9 schon „diverged“ schreiben. `git status` kennt Kims Commit erst nach `git fetch`; bis dahin sieht es nur `origin/main` auf A. In (a) `origin/main` auf R setzen: Der Push in Zeile 12 kam vor dem Revert.

Bewertung: (a) Commits und Eltern 1 P, Merge-Commits mit erstem Elternteil 1 P, alle Zeiger 1 P. (b) drei Merges 1 P, `fix/illinois` mit Begründung 1 P. (c) vier Zeilen der Tabelle 1 P, Begründung zu Zeile 9 1 P. (d) Commits, oberster und Merge-Commits 1 P, `fix/illinois` und `--all` 1 P.

Lösung zu Aufgabe 2 Dreiwege-Merge von Hand

(a)

Stelle	geändert von	Ergebnis
`description`	beiden, gleich	automatisch, „für“
`requires-python`	nur ihrer Seite	automatisch, `">=3.13,<3.14"`
Liste `dev` nach `mypy`	beiden, verschieden	Konflikt
`[tool.pytest.ini_options]` am Ende	nur unserer Seite	automatisch, angehängt

Git vergleicht jede Seite mit der Basis. Wer allein geändert hat, gewinnt; gleiche Änderungen sind kein Konflikt. In der Liste fügen beide an derselben Stelle (zwischen `mypy` und `ruff`) Verschiedenes ein, also entscheidet Git nicht.

(b) Oben steht unsere Seite (`HEAD`, also `main`), unten ihre mit dem Namen des Branches. Die übernommenen Änderungen stehen schon ohne Marker in der Datei: 18 Zeilen, davon 3 Marker.

```
description = "Importer für beide Werke"
readme = "README.md"
requires-python = ">=3.13,<3.14"
dependencies = []

[dependency-groups]
dev = [
    "mypy>=2.4.0",
<<<<<<< HEAD
    "pytest>=9.1.1",
=======
    "pre-commit>=4.7.0",
>>>>>>> feature/hooks
    "ruff>=0.17.0",
]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

(c) Fachlich wollen beide etwas Neues in derselben Liste, und beides wird gebraucht: pytest für die Tests, pre-commit für die Hooks. Richtig sind beide Zeilen, alphabetisch wie bei uv: `"mypy>=2.4.0"`, `"pre-commit>=4.7.0"`, `"pytest>=9.1.1"`, `"ruff>=0.17.0"`. Nimmst du nur eine Seite, fehlt nach dem Merge ein Werkzeug, und die CI oder pre-commit scheitert.

Für `uv.lock` ist Weg (3) richtig. `uv.lock` schreibt uv; mit Markern kann `uv lock` die Datei nicht einmal lesen und bricht mit „Failed to parse `uv.lock`“ ab. Von Hand (1) entsteht leicht ein Lockfile, das nicht zu `pyproject.toml` passt. Löschen (2) löst alles neu auf; uv nimmt dann die neuesten erlaubten Versionen aller Pakete, und die festgehaltenen Versionen sind weg. Mit (3) behält uv die Versionen von `main` und ergänzt nur pre-commit und seine Abhängigkeiten. Abschluss:

```
git checkout --ours uv.lock
uv lock
uv sync --locked
uv run pytest
git add pyproject.toml uv.lock
git commit --no-edit
```

Typischer Fehler

In (a) die gleiche Korrektur von „fuer“ für einen Konflikt halten. Beide Seiten wollen dasselbe, Git übernimmt es ohne Rückfrage.

Bewertung: (a) vier Stellen richtig eingeordnet 2 P, bei drei Stellen die Hälfte. (b) Konfliktblock mit Markern und Namen 1 P, übernommene Änderungen außerhalb 1 P. (c) beide Werkzeuge mit Begründung 1 P, Weg (3) mit Begründung gegen (1) und (2) 1 P, Befehle zum Abschluss 1 P.

Lösung zu Aufgabe 3 Tests entwerfen und bewerten

(a) Äquivalenzklassen: Früh (06:00 bis 13:59), Spät (14:00 bis 21:59), außerhalb der Schichten (00:00 bis 05:59 und 22:00 bis 23:59, beide mit „keine Schicht“) und unlesbar (etwa `24:00` oder `6:00`). Grenzen liegen bei 06:00, 14:00 und 22:00; jede prüfst du von beiden Seiten, das sind 6 Uhrzeiten:

Eingabe	`05:59`	`06:00`	`13:59`	`14:00`	`21:59`	`22:00`	`24:00`
erwartet	keine Schicht	Früh	Früh	Spät	Spät	keine Schicht	unlesbar

Die sechs Grenzwerte decken die drei gültigen Klassen schon ab, `24:00` kommt für die unlesbare Klasse dazu: 7 Eingaben. „keine Schicht“ prüfst du mit `match`, damit der Fall sich von „unlesbar“ unterscheidet.

(b) Rot heißt: Der Test fängt den Mutanten.

	T1 `09:30`	T2 `14:00`	T3 `21:59`	T4 `23:10`	T5 `05:59`	Ergebnis
M1	grün	grün	grün	grün	grün	überlebt
M2	grün	rot	grün	grün	grün	gefangen
M3	grün	grün	grün	grün	grün	überlebt
M4	grün	grün	grün	grün	rot	gefangen
M5	rot	rot	rot	grün	rot	gefangen

M1 macht aus 06:00 „keine Schicht“, aber kein Test fragt 06:00 ab: Es fehlt `("06:00", "Früh")` in `test_schicht`. M3 macht aus 22:00 „Spät“: Es fehlt ein Test, dass `22:00` einen `ValueError` mit `match="keine Schicht"` auslöst. M2 fängt nur T2, denn nur 14:00 liegt auf der geänderten Grenze. M4 ändert nur die Meldung; das sieht allein T5 mit `match`. Unter M5 scheitert

`strptime` an jedem `HH:MM` mit einem `ValueError`. T4 verlangt nur irgendeinen `ValueError` und bleibt grün: ohne `match` zu großzügig. T5 sieht die falsche Meldung und wird rot.

(c) Der erste Test vergleicht, wirft das Ergebnis aber weg; ohne `assert` prüft er nichts. Der zweite prüft nur, dass das Ergebnis wahr ist, also kein leerer Text; `"Früh"` ist nur die Meldung für den Fehlerfall. Rot werden sie nur, wenn `schicht` eine Ausnahme wirft, der zweite auch bei leerem Ergebnis. Eine Fassung, die immer `return "Nacht"` liefert, lässt beide grün. Richtig:

```
def test_spaet_am_abend():
    assert schicht("17:45") == "Spät"


def test_frueh_am_morgen():
    assert schicht("07:15") == "Früh"
```

Gegenprobe: Erwartung kurz ändern, der Test muss rot werden. Den ersten meldet schon `ruff check` mit seinen voreingestellten Regeln (B015, ein Vergleich ohne Wirkung), den zweiten nicht; pre-commit und CI hätten nur den ersten gestoppt.

(d) pytest führt die Tests einer Datei in ihrer Reihenfolge aus. Mit `scope="module"` entsteht die Liste einmal für die ganze Datei, und der erste Test verkleinert sie.

Aufruf	`test_nur_schichtzeiten`	`test_drei_meldungen`
(1) beide Tests	grün	rot, `assert 2 == 3`
(2) mit `-k drei`	nicht ausgewählt	grün
(3) ohne `scope="module"`	grün	grün

Derselbe Test ist also je nach Auswahl grün oder rot. Unabhängig wird er mit dem voreingestellten Gültigkeitsbereich `function`, bei dem jeder Test eine frische Liste bekommt, oder wenn kein Test die Fixture verändert, etwa `[m for m in meldungen if m != "05:40"]`.

Typischer Fehler

In (b) glauben, T4 fange M5, weil unter M5 doch ein Fehler kommt. Ohne `match` ist `pytest.raises(ValueError)` mit jedem `ValueError` zufrieden, auch mit dem falschen.

Typischer Fehler

In (a) nur Werte mitten in den Klassen nehmen, etwa 09:30 und 17:00. Fehler wie `<` statt `<=` sitzen an den Grenzen; dort findet sie nur ein Paar wie 05:59 und 06:00.

Bewertung: (a) Klassen 1 P, Grenzwerte mit Ergebnis 1 P. (b) Tabelle 2 P (mit einer falschen Zeile die Hälfte), fehlende Tests 1 P, T4 und `match` 1 P. (c) Begründung beider Tests 1 P, falsche Fassung und richtige Tests 1 P. (d) drei Aufrufe 1 P, Lösung 1 P.

Lösung zu Aufgabe 4 CI-Protokoll und Review

(a) Rot ist der Schritt `uv run pytest`. Es scheitert der Test `test_vor_sechs_keine_schicht` in der Datei `tests/test_schicht.py` in Zeile 26, dort steht `with pytest.raises(...)`. Er erwartet, dass `schicht("05:59")` einen `ValueError` mit „keine Schicht“ auslöst; seit der Änderung liefert die Funktion aber „Früh“, pytest meldet „DID NOT RAISE“. Die vier anderen Tests sind grün. Die Schritte davor liefen alle und waren grün: Formatierung, ruff, mypy. Eine CI bricht beim ersten roten Schritt ab; bei rotem `ruff check` wäre `uv run pytest` nicht gelaufen.

Zu tun: Hier ist nicht der Code falsch, sondern die alte Erwartung, denn die Werkleitung hat die Regel geändert. Kim passt den Test an die neue Regel an, statt ihn zu löschen: 05:59 ist jetzt Früh, die neue Grenze prüfen `05:29` (keine Schicht) und `05:30` (Früh). Lokal `uv run pytest`, dann committen und pushen; der Pull Request nimmt den Commit auf, und die CI läuft neu.

(b) Zum Beispiel (gleichwertige Formulierungen zählen):

Stelle	Kommentar
`ci.yml`, `uv sync`	„Magst du die Zeile wieder auf `uv sync --locked` stellen? Mit `--locked` prüft die CI, dass `uv.lock` zu `pyproject.toml` passt, und testet genau die Versionen von unseren Laptops. Mit der Frühschicht hat die Änderung nichts zu tun.“
`schicht.py`, `print`	„Das `print` sieht nach einem Rest vom Ausprobieren aus. In Bibliothekscode erscheint es bei jedem Aufruf, und niemand kann es abschalten. Kannst du es entfernen oder, falls du es brauchst, als `log.debug` schreiben?“
`schicht.py`, Docstring	„Der Docstring nennt noch Früh ab 06:00. Magst du ihn auf 05:30 ändern? Die Definition of Done verlangt angepasste Docstrings, wenn sich das Verhalten ändert, und die Tests lesen ihn als Regel.“
`test_schicht.py`	„Der gelöschte Test war rot, weil 05:59 jetzt Früh ist. Statt ihn zu löschen: Kannst du `("05:59", "Früh")` und `("05:30", "Früh")` in `test_schicht` aufnehmen und den Test für keine Schicht mit `05:29` behalten? Dann ist die neue Grenze von beiden Seiten geprüft.“

Ein Kommentar zählt, wenn er die Stelle nennt, das Problem begründet und eine Änderung vorschlägt, als Frage oder Vorschlag statt als Befehl. Ein kurzes Lob gehört dazu, etwa für die klare Beschreibung mit „Fixes #4“. Andere begründete Probleme zählen ebenso, etwa dass `schicht` kein Datum kennt und frühere Meldungen zwischen 5:30 und 5:59 jetzt als Frühschicht zählen, obwohl die Regel erst ab dem 2. November gilt.

> **Typischer Fehler**
>
> In (b) das Löschen des Tests als Lösung akzeptieren, weil die CI jetzt grün ist. Grün heißt nur, dass alle vorhandenen Tests bestehen. Ändert sich eine Regel, ändert sich der Test mit, und die neue Grenze braucht eigene Fälle.

Bewertung: (a) Schritt, Test, Datei und Zeile 1 P, Erwartung, Ergebnis und Abbruch beim ersten roten Schritt 1 P, was zu tun ist 1 P. (b) erstes und zweites Problem mit freundlichem, begründetem Vorschlag 2 P, drittes und viertes 2 P; ein Kommentar ohne Begründung oder im Befehlston zählt halb.

Lösung zu Aufgabe 5 Versionen und Rückgängig machen

(a) Die erste Stelle steigt bei einem Bruch, die zweite bei neuen Funktionen, die dritte bei Fehlerbehebungen; was rechts davon steht, fällt auf 0.

Änderung	Art	Version	Befehl
(1) Summenzeile	Fehlerbehebung	`1.3.3`	`uv version --bump patch`
(2) Unterbefehl	neue Funktion, kein Bruch	`1.4.0`	`uv version --bump minor`
(3) Option umbenannt	Bruch	`2.0.0`	`uv version --bump major`
(4) (1) und (2)	größte Änderung zählt	`1.4.0`	`uv version --bump minor`

Die Kommandozeile ist die Schnittstelle von `nordwerk`: Jeder Aufruf mit `--aus`, etwa in einem Skript oder in der CI, scheitert nach der Umbenennung. Mit einem neuen Unterbefehl ändert sich für bisherige Aufrufe nichts.

(b)

Befehl	`main`	vorne	hinten	`ausgang`	Zeile aus C	Änderung in `cli.py`
(1) `restore`	C	0	0	`export`	committet	verloren
(2) `revert`	R, neu	1	0	`daten/aufbereitet`	committet	bleibt, nicht vorgemerkt
(3) `reset --soft`	B	0	1	`export`	vorgemerkt	bleibt, nicht vorgemerkt
(4) `reset --hard`	A	0	2	`daten/aufbereitet`	weg	verloren

R ist ein neuer Commit nach C, der B umkehrt; die Geschichte A, B, C bleibt. `reset --soft` nimmt nur den Commit C weg, seine Änderung liegt vorgemerkt bereit für einen neuen Commit. `reset --hard` setzt Branch, Staging-Bereich und Arbeitsverzeichnis auf A und wirft dabei auch die Änderung in `cli.py` weg, die nie committet war; die ist nicht wiederherzustellen. Nach dem Push sicher sind (1) und (2): `restore` betrifft nur Ungespeichertes auf deinem Laptop, `revert` hängt nur einen Commit an. (3) und (4) entfernen gepushte Commits aus `main`; ein normaler Push danach wird abgelehnt, und `--force` nähme B und C allen anderen weg.

Typischer Fehler

In (a) bei (3) `2.3.2` schreiben. Steigt eine Stelle, beginnen alle Stellen rechts davon wieder bei 0.

Typischer Fehler

In (b) glauben, `reset --hard` betreffe nur Commits. Es setzt auch das Arbeitsverzeichnis zurück; nicht committete Änderungen sind danach weg. Schau vorher mit `git status` und `git diff`.

Bewertung: (a) (1) bis (3) 2 P, (4) und Befehle 1 P. (b) Zeiger, vorne und hinten 1 P, ausgang und Zeile aus C 1 P, `cli.py` 1 P, sichere Befehle mit Begründung 1 P.